Event Module

Thapa Technical

Event Module - Node.js

EventEmitter is a core module in Node.js used to create and handle custom events.

It is part of the events module and is often used for building event-driven systems in Node.js.

Key Methods

1. emit(eventName, [args])

Purpose: Emits (or triggers) an event with the specified eventName. You can also pass arguments that will be consumed by the listeners.

It's like calling a function, but instead, it triggers all listeners (functions) attached to the specified event.

2. on(eventName, listener)

Purpose: Attaches a listener (a function) to a specific eventName. This listener will execute when the event is emitted.

Event Module - Node.js

Think of it as defining custom events and then triggering them:

Defining a Listener (on):

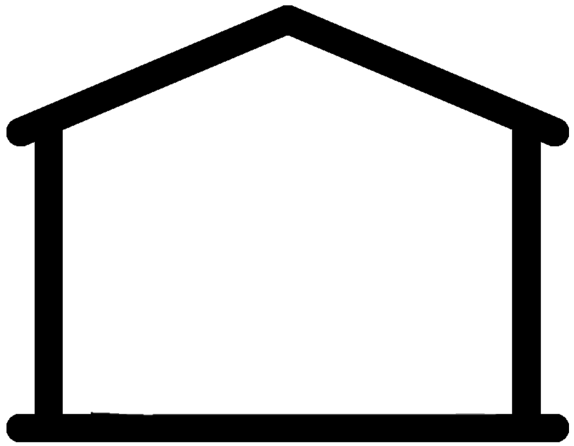
It's like defining a function for an event.

The listener will execute when the corresponding event is triggered.

Emitting an Event (emit):

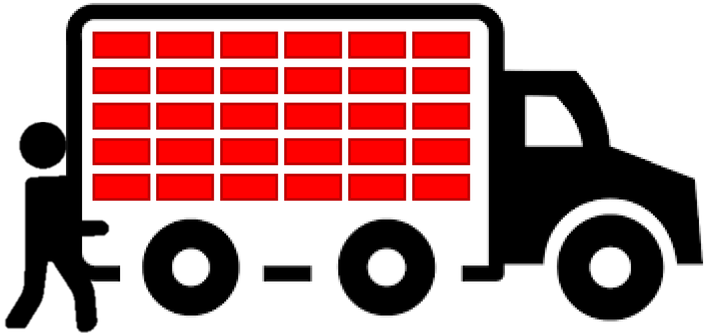
It's like calling that event's listener.

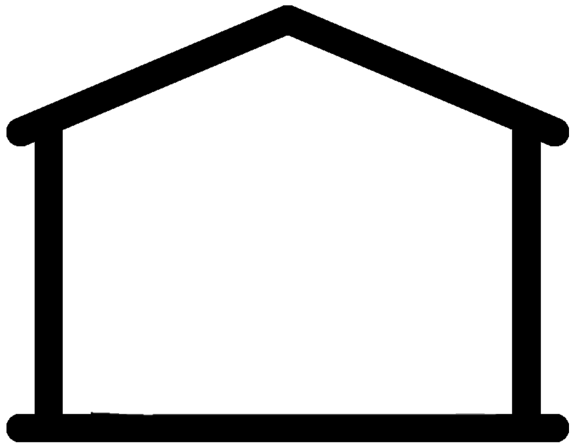
Streams and Buffers



Imagine, you have a truck full of bricks that you want to carry.

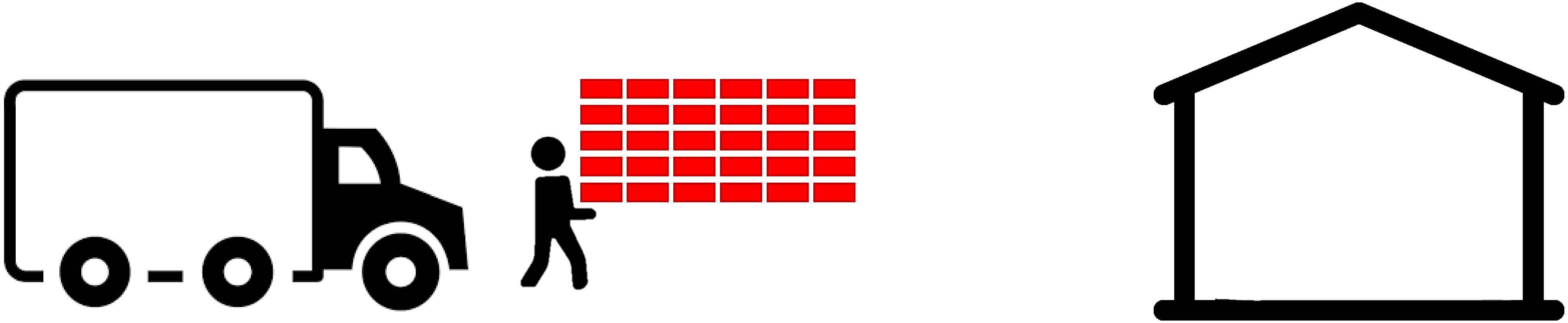
Streams and Buffers





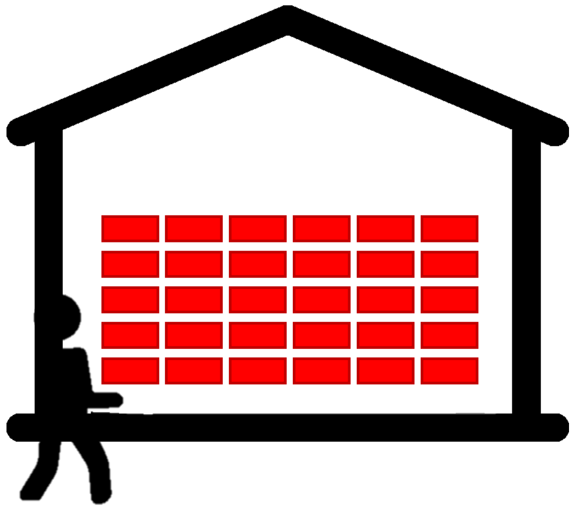
You can try to carry everything at once

Streams and Buffers



You can try to carry everything at once

Streams and Buffers



You can try to carry everything at once

Streams and Buffers

You can try to carry everything at once

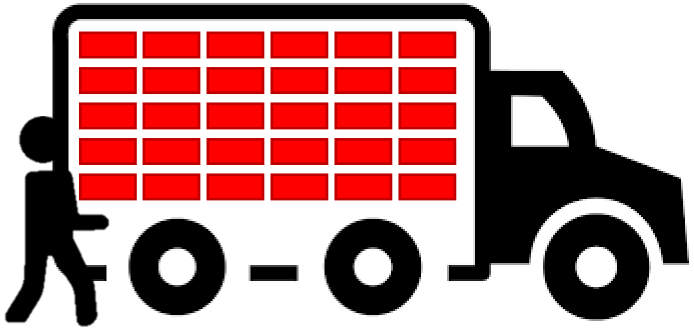
Streams and Buffers

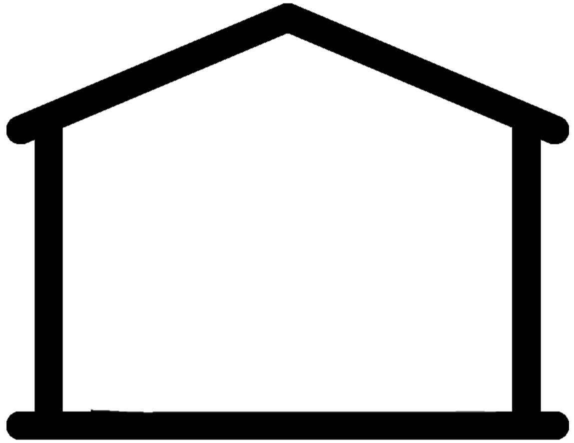
But is this possible? Maybe, if you are a gorilla, but it’s not possible for a normal human being.

Streams and Buffers

So, what you will typically do is, take small chunk of bricks one by one and take it.

Streams and Buffers



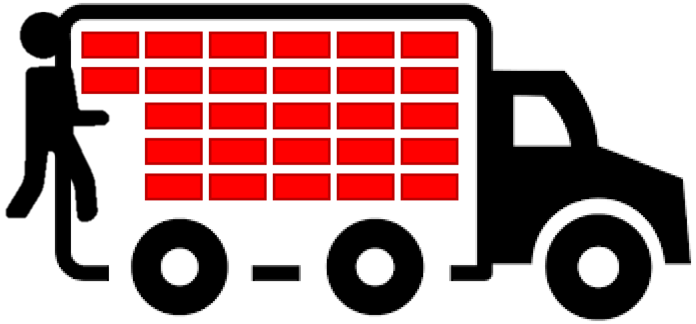


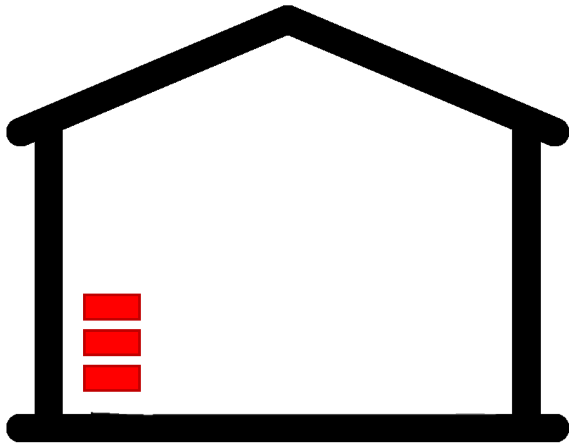
So, what you will typically do is, take small chunk of bricks one by one and take it.

Streams and Buffers

So, what you will typically do is, take small chunk of bricks one by one and take it.

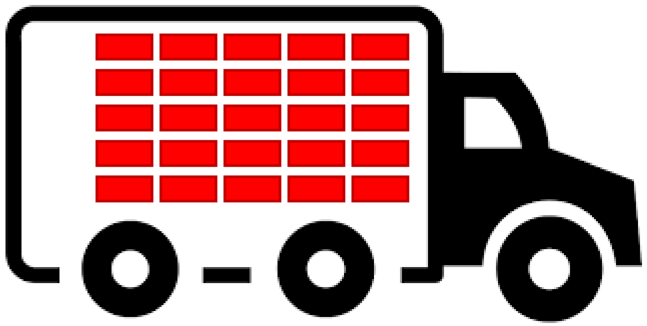
Streams and Buffers

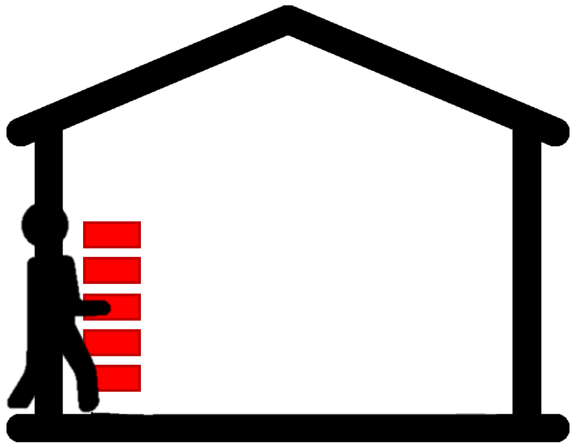




So, what you will typically do is, take small chunk of bricks one by one and take it.

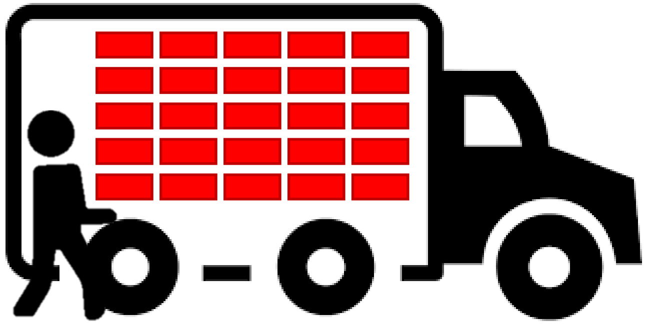
Streams and Buffers

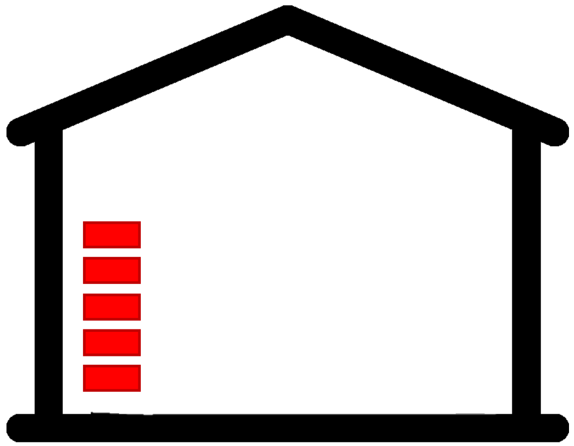




So, what you will typically do is, take small chunk of bricks one by one and take it.

Streams and Buffers





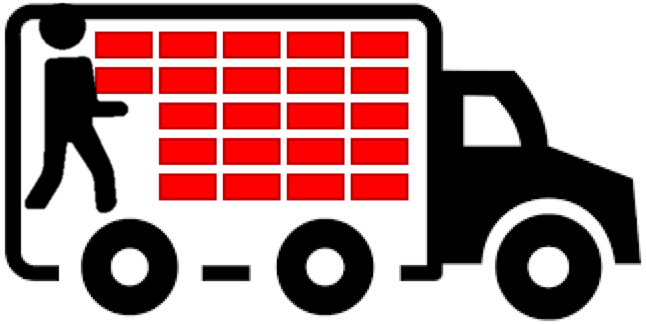
THAPA TECHNICAL – NODEJS YOUTBE SERIES

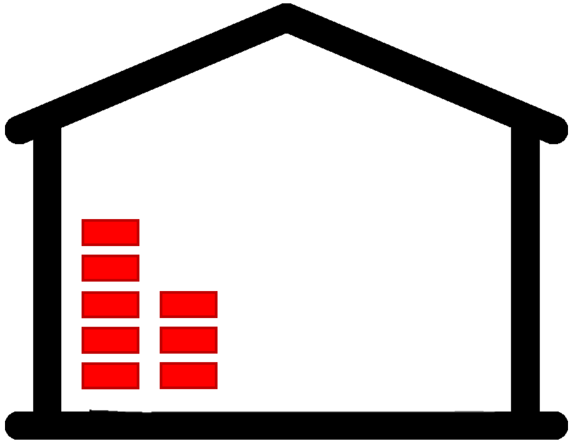
So, what you will typically do is, take small chunk of bricks one by one and take it.

Streams and Buffers

So, what you will typically do is, take small chunk of bricks one by one and take it.

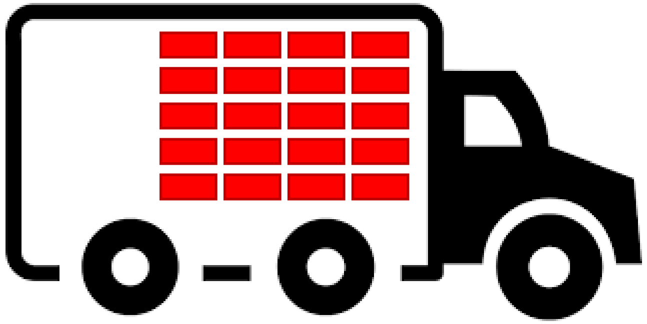
Streams and Buffers





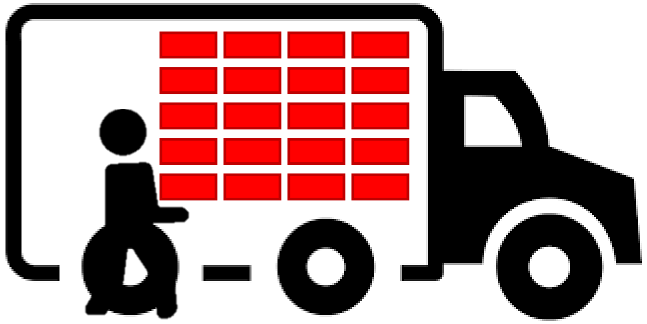
So, what you will typically do is, take small chunk of bricks one by one and take it.

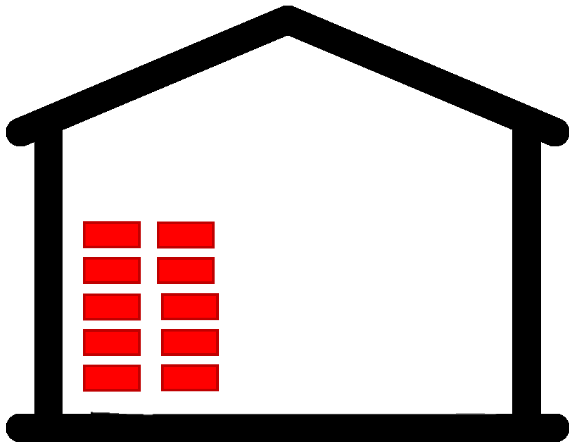
Streams and Buffers



So, what you will typically do is, take small chunk of bricks one by one and take it.

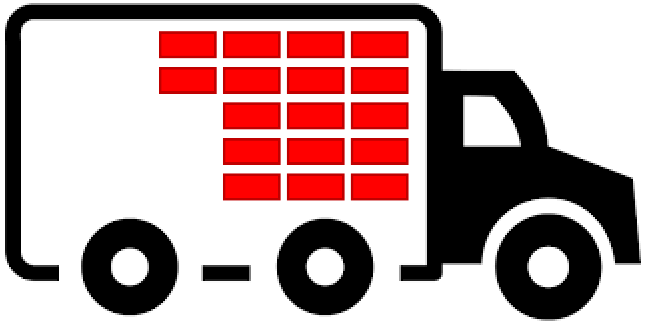
Streams and Buffers





So, what you will typically do is, take small chunk of bricks one by one and take it.

Streams and Buffers

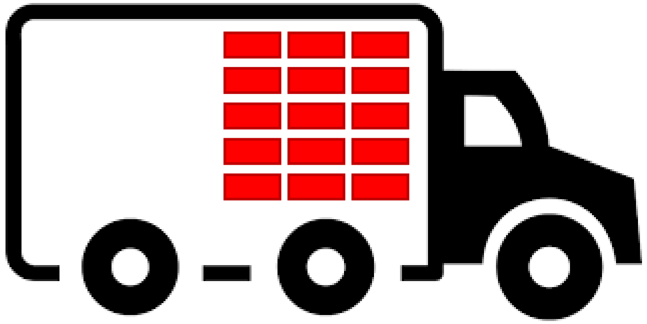


So, what you will typically do is, take small chunk of bricks one by one and take it.

Streams and Buffers

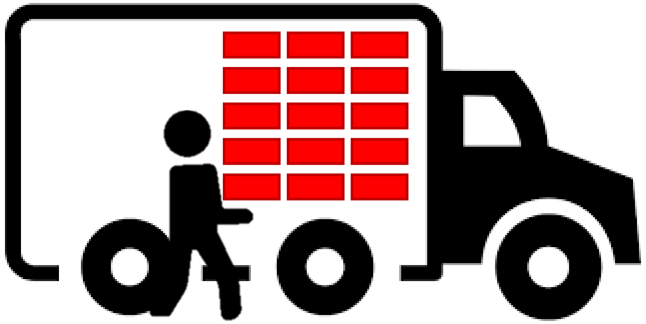
So, what you will typically do is, take small chunk of bricks one by one and take it.

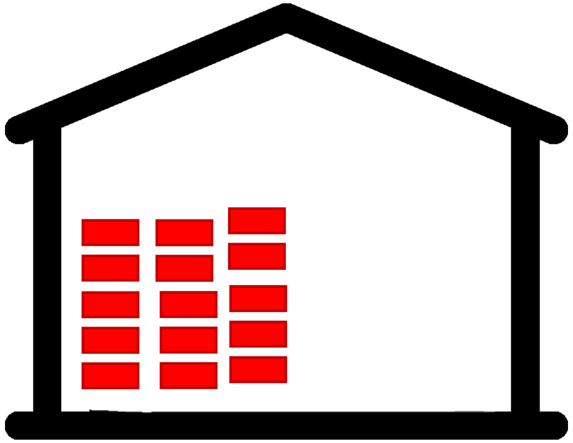
Streams and Buffers



So, what you will typically do is, take small chunk of bricks one by one and take it.

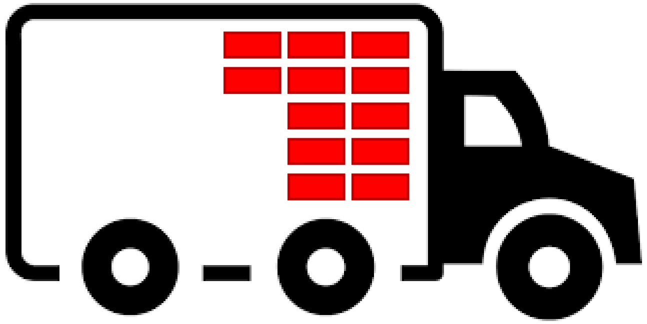
Streams and Buffers





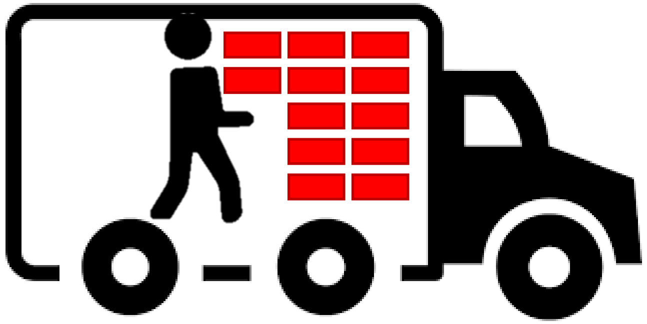
So, what you will typically do is, take small chunk of bricks one by one and take it.

Streams and Buffers



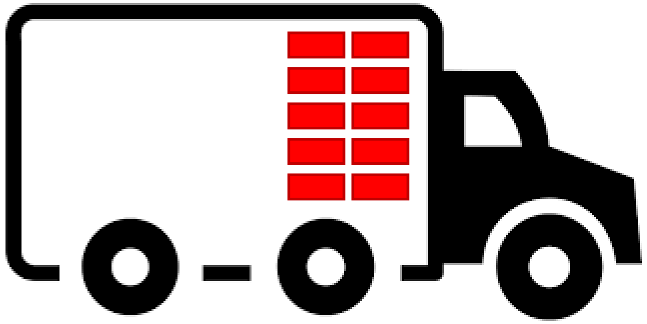
So, what you will typically do is, take small chunk of bricks one by one and take it.

Streams and Buffers



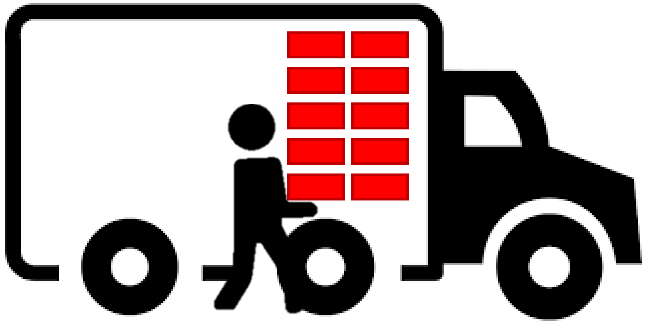
So, what you will typically do is, take small chunk of bricks one by one and take it.

Streams and Buffers



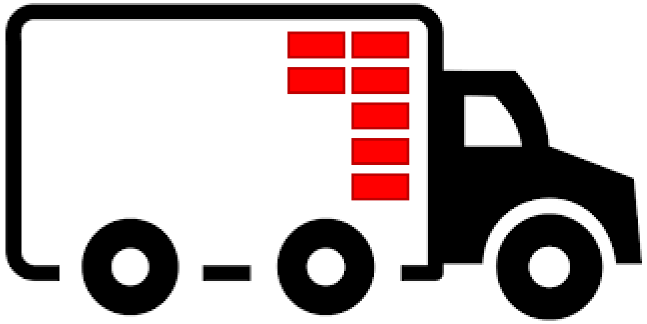
So, what you will typically do is, take small chunk of bricks one by one and take it.

Streams and Buffers



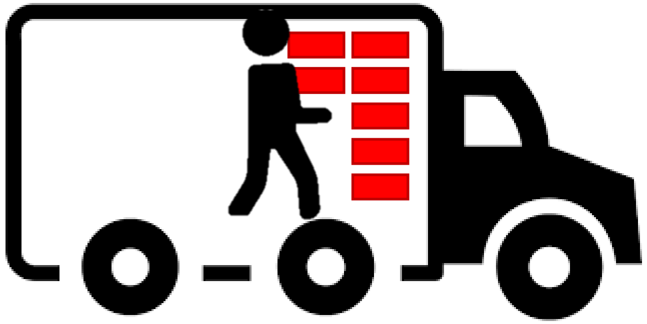
So, what you will typically do is, take small chunk of bricks one by one and take it.

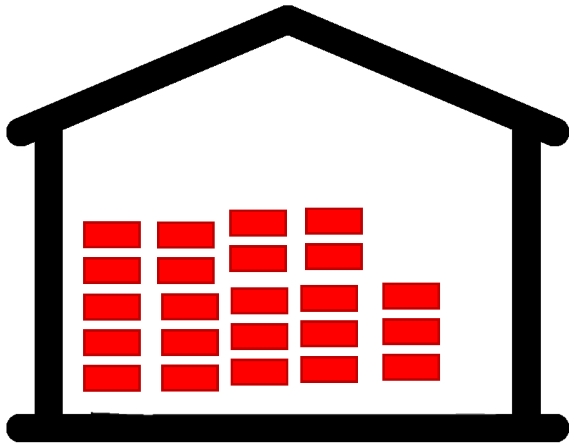
Streams and Buffers



So, what you will typically do is, take small chunk of bricks one by one and take it.

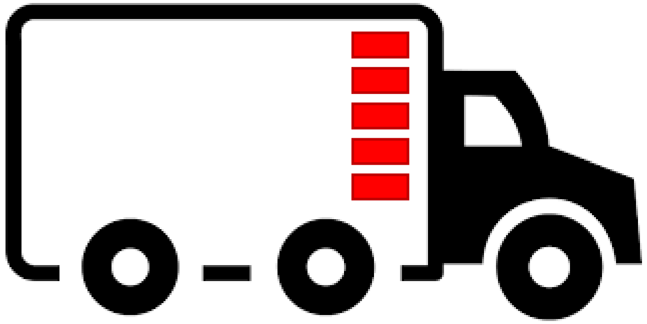
Streams and Buffers





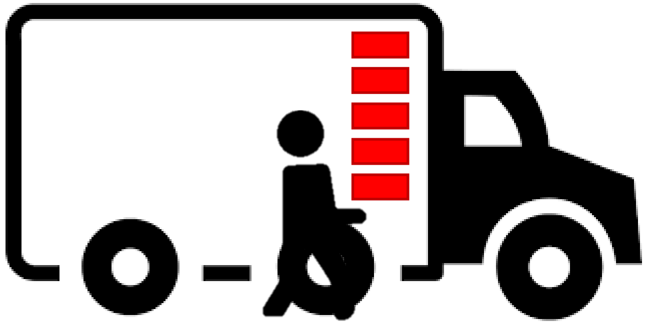
So, what you will typically do is, take small chunk of bricks one by one and take it.

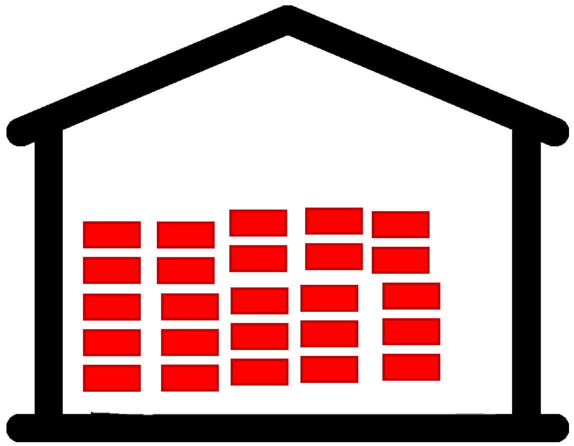
Streams and Buffers



So, what you will typically do is, take small chunk of bricks one by one and take it.

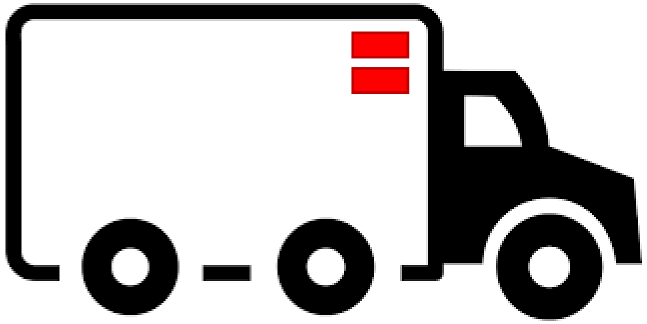
Streams and Buffers





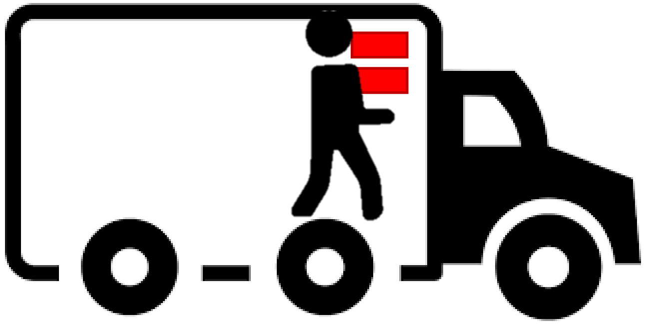
So, what you will typically do is, take small chunk of bricks one by one and take it.

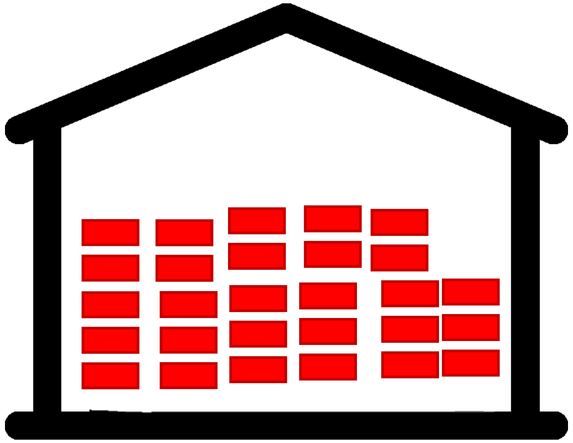
Streams and Buffers



So, what you will typically do is, take small chunk of bricks one by one and take it.

Streams and Buffers



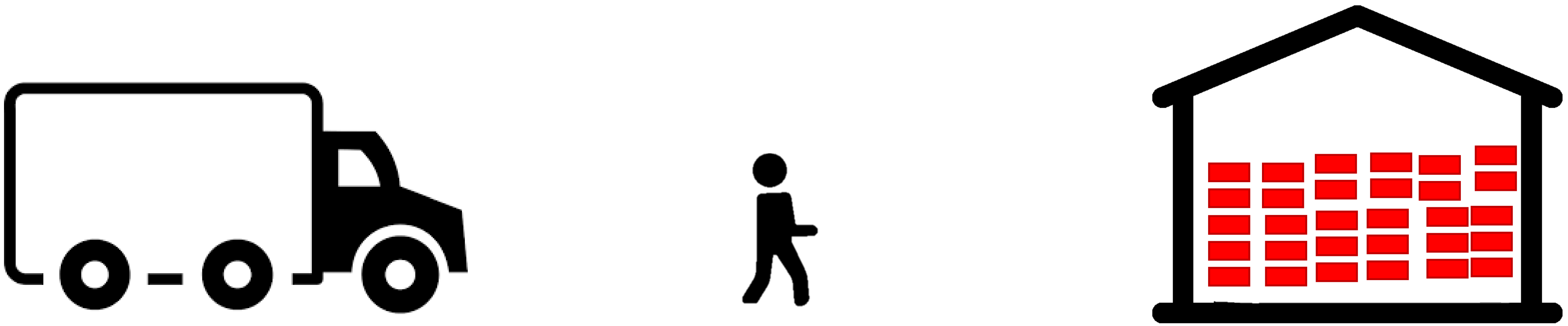


So, what you will typically do is, take small chunk of bricks one by one and take it.

Streams and Buffers

So, what you will typically do is, take small chunk of bricks one by one and take it.

Streams and Buffers



Finally completed, but how is this related to Node.js?

Streams and Buffers

8 GB RAM

16 GB RAM

Let’s, change the example to client-server relation.
Let’s imagine a client has 8 GB RAM, and server has 16 GB RAM.

Streams and Buffers

If a client wants to upload a 2 TB file to the server, then what it can do is give whole 2 TB file to server, Then server can store it in disk. But is that possible?

Streams and Buffers

It’s not possible because client first has to take whole 2 TB file in their RAM, then upload it to server, then server should initially store whole 2 TB file in RAM, then store it in disk.

Streams and Buffers

Neither client has 2 TB RAM, nor the server can handle it. In that case, we have to split the file
Into chunks and server should take the file in chunk and write into disk at the same time.

Streams and Buffers

16 GB RAM

8 GB RAM

2TB File

For that, we use streams and buffers in Node.js

Streams and Buffers

- Streams in Node.js are a way to handle continuous flow of data.
- They enable reading or writing data piece by piece instead of loading the entire data into memory.
- A Buffer in Node.js is a temporary storage area for binary data.
- Buffers work as chunks of data, enabling efficient data manipulation in streams.
- It helps to decrease memory consumption of your web server.
- There are four types of streams in Node.js: Writeable, Readable, Duplex, and Transform. But we won't cover everything in detail, as that would take a separate course if we wanted. We will teach what is needed for now.